Отчёт об очистке репозитория

Дата: 26.09.2026. Репозиторий `Pavel1778/rtk-crm`, ветка `dev`, коммит `232ad7a`.

Итог

Очистка истории не требуется. Проверка показала, что персональных данных, секретов и мусорных артефактов в истории нет, поэтому перезапись истории (`git filter-repo`/BFG) не выполнялась: она бы изменила хеши 185 коммитов и потребовала переклонирования у всех участников без какой-либо пользы.

Что проверено

Проверка	Команда	Результат
`.env`, `.pem`, `.key`, тории	`git log --all --full-history -- '*.env*' '*.pem' '*.key' '*.db' '*.sqlite*'`	только `backend/.env.example`, `frontend/.env.example`, `infra/yandex-cloud/.env.prod.example` — шаблоны с плейсхолдерами
Реальные значения секретов	`git log --all -p -- '*.env*' | grep -iE '(SECRET_KEY|PASSWORD|TOKEN|API_KEY)='`	совпадения только с пустыми значениями и закомментированными строками
Приватные ключи	поиск `BEGIN ... PRIVATE KEY` по всей истории	0
Токены Supabase/GigaChat	поиск `eyJ...` и `service_role`	0
Файлы с ПДн кейсодержателя	`git log --all --full-history -- "Загрузка пользователей.xlsx" "Данные оплат.json" "Вендоры.xlsx"`	в истории отсутствуют
Персональные данные в истории	поиск СНИЛС и паспортных шаблонов	только синтетический плейсхолдер `123-456-789 00` в тестах
Мусор в индексе	`git ls-files | grep -iE 'node_modules|__pycache__|dist/|build/|.venv|.idea|.DS_Store'`	пусто
Крупные файлы	`git rev-list --objects --all | git cat-file --batch-check | sort`	максимум 1.91 МБ (`docs/presentation/template-lct2026.pptx`), порог 50 МБ не достигается
Следы сторонних инструментов	`git log --all --format='%b'` на предмет трейлеров авторства и подписей генераторов	пусто
Конфиги сторонних ассистентов	`git ls-files` на файлы конфигурации ассистентов в корне и подкаталогах	пусто
Абсолютные пути среды сборки	поиск `home/<build-user>` и доменов предпросмотра	только в игнорируемых кэшах, удалены; в отслеживаемых файлах пусто

Уточнение по ранее удалённым значениям

В `docs/VERIFICATION_REPORT.md` (пункт про старые секреты) отмечено, что ранее опубликованные учётные данные нельзя считать отозванными. Проверка истории уточняет эту оценку:

- **Реальных учётных данных третьих сторон в истории нет.** Поиск по всем 952 blob-объектам не нашёл ни настоящих хостов Supabase (вида `<project-ref>.supabase.co`), ни JWT-токенов, ни ключей доступа GigaChat. В истории встречаются только placeholder'ы: `<host>.pooler.supabase.com`, `<service>.onrender.com`, `<password>`, `<cluster-id>`.
- **Что действительно было в репозитории** — слабые значения по умолчанию для локального запуска, удалённые в `9643426`:

Значение	Где было	Риск
`SECRET_KEY: change-me-32-chars-minimum`	`docker-compose.yml`	подделка JWT, если контур поднят без переопределения
`SECRET_KEY: rtk-crm-2026-hackathon-secret-key-32chars`	`docs/DEPLOYMENT.md`	то же
`POSTGRES_PASSWORD: rtk_password`	`docker-compose.yml`	доступ к БД

Все они относятся к локальному контуру (`postgres:5432`, `localhost`), а не к облачным сервисам.

Что из этого следует. Ротация сторонних ключей (Supabase, Render) не требуется: их в репозитории не было. Но если контур разворачивался из `docker-compose.yml` без переопределения `SECRET_KEY`, то значение `change-me-32-chars-minimum` попало в работающий сервис, и его нужно заменить: подпись JWT на таком ключе подделывается тривиально. Текущий код этому препятствует — в production `Settings` отклоняет дефолтный и слишком короткий ключ, а Compose требует его явно (`SECRET_KEY: ${SECRET_KEY:?SECRET_KEY is required}`).

Синтетические данные вместо ПДн

Наборы для импорта собираются в памяти тестами (`tests/test_excel_import.py`, `tests/test_import_report.py`) и содержат заведомо вымышленные значения. Тест `test_excel_import.py` отдельно проверяет, что импорт не сохраняет поля СНИЛС, паспорта и адреса: чувствительные колонки отбрасываются на этапе разбора. Поэтому отдельные обезличенные копии в `docs/examples/` не нужны — их роль выполняют фикстуры и примеры отчётов (`docs/examples/README.md`).

Что сделано

1. `.gitignore` дополнен шаблонами для ключей и сертификатов: `*.pem`, `*.key`, `credentials.json`, `service-account.json`, `service_account.json`, `key.json`, `*.p12`, `*.pfx`. Ранее эти файлы не были перечислены явно, хотя попасть в репозиторий не могли — их просто не создавали. Паттерны проверены через `git check-ignore`: отслеживаемые файлы не затронуты.
2. Добавлен `docs/examples/README.md` с объяснением происхождения данных.

Состояние репозитория

Показатель	Значение
Коммитов	185
Отслеживаемых файлов	276
Размер `.git`	19 МБ
Размер рабочих файлов	15 МБ
Крупнейший файл в истории	1.91 МБ

Вывод

Риск утечки ПДн через историю репозитория отсутствует. Дополнительная перезапись истории не нужна и была бы вредна: она ломает подписи коммитов, требует force-push в `main` и вынуждает всех участников переклонировать репозиторий.